Retrieval-Augmented Generation: A Primer

The problem RAG solves

A large language model answers from what it memorized during training. It cannot see documents that were uploaded after training, and it can invent plausible-sounding facts when asked about something it never learned. Retrieval-augmented generation, usually shortened to RAG, fixes both issues by letting the model quote a source that is attached to the question at run time.

How a RAG pipeline works

A RAG pipeline has five stages. First, ingestion: an uploaded document is parsed into plain text. Second, chunking: the text is cut into overlapping pieces, commonly a few hundred characters each, so no single piece is too long for the embedding model. Third, embedding: every chunk is converted into a vector of numbers such that chunks about similar topics land near each other in vector space. Fourth, retrieval: the user's question is embedded the same way and the chunk vectors closest to the question vector are fetched from a vector index. Fifth, generation: the question and the retrieved chunks are sent to the language model, which is instructed to answer only from that context.

Why chunk size matters

If a chunk is too large, it may contain several unrelated topics and the embedding becomes a blurry average, which hurts retrieval precision. If a chunk is too small, it may not contain the full answer even when it is the closest match, which hurts retrieval recall. Overlap between consecutive chunks reduces the chance that a fact sitting across a cut boundary is lost entirely.

Measuring retrieval quality

Retrieval quality is measured with a ground-truth fixture: a list of questions paired with the chunks that should be retrieved. Hit rate at k is the fraction of questions for which at least one gold chunk appears in the top k retrieved chunks. Recall at k is the fraction of all gold chunks that appear in the top k. Recall is the stricter of the two whenever a question has more than one gold chunk.

Measuring answer faithfulness

A retrieved chunk can be relevant and the answer still wrong. Faithfulness asks a narrower question: is every claim in the generated answer supported by the retrieved context? A common way to score it is LLM-as-judge: a second language model call receives the question, the answer, and the context, and returns a verdict of faithful, partial, or unfaithful. An answer that states something the context does not contain is unfaithful even if it happens to be true.

Limits of RAG evaluation

A small fixture can only demonstrate that the pipeline works end to end; it cannot prove the system is accurate in general. Hit rate and recall are sensitive to how gold chunks were chosen, and an LLM judge inherits the biases of the judge model. For these reasons, evaluation numbers should always be reported together with the size and origin of the fixture they were measured on.